SISTEMAS OPERATIVOS I

Práctica 2: Gestión de procesos

Objetivo

Manejar herramientas básicas de creación y terminación de procesos, y comprensión de compartición de memoria entre procesos padre e hijo

Información útil para la práctica

- Busca información sobre las variables de entorno y su visualización en un proceso con la función *getenv*.
- Busca información sobre las llamadas al sistema *getpid*, *getppid*, *getuid*, *geteuid* y *getgid*.
- Las variables que se utilicen como *pid* de un proceso deben definirse como tipo *pid_t*, en caso contrario la compilación generará un *warning*. En todo caso, su utilización en el programa es como un entero.
- Recuerda que *fork* crea un proceso que es exactamente igual al proceso padre salvo en su *PID*. El proceso hijo se crea como una copia del proceso padre, incluyendo su espacio de direcciones y el contenido de los registros, pero no comparten su espacio de memoria.
- La terminación de los procesos se debe hacer con la llamada al sistema *exit*.
- Se recomienda usar el comando *ps* desde un terminal adicional para comprobar la presencia de todos los procesos que se crean y su estado. Para mantener a los procesos activos mientras se usa el comando *ps*, se recomienda utilizar llamadas a la función *sleep* o bien *scanf* o *getchar* para mantener a los procesos esperando el tiempo necesario.
- Con la llamada al sistema *wait* no es posible especificar el hijo concreto por el que espera el padre. La alternativa para ello es usar *wait*. Recuerda que para el padre salga del *wait*, el hijo debe estar en estado zombie.

Enunciado

1. Haz un programa para crear un proceso hijo usando la función de POSIX *fork*, y que ofrezca información identificativa tanto del proceso padre como del proceso hijo utilizando las llamadas al sistema *getpid*, *getppid*, *getuid*, *geteuid*, *getgid* y *getenv*.
2. En el programa anterior, incluir comprobaciones de que las direcciones de memoria de las variables (globales, locales y dinámicas) del padre y del hijo son las mismas "virtualmente", pero que son realmente variables "físicamente" diferentes (los valores de las variables son distintos). Comprueba el valor de la posición de memoria (puntero) que ocupan. Puedes considerar una determinada posición de memoria comprobando que inicialmente tiene el mismo contenido para los dos procesos, pero que si uno de ellos lo modifica, el otro proceso mantiene su valor (el original) porque realmente la variable es otra.
3. En el programa anterior comprueba que si el padre abre un fichero antes de ejecutar el *fork*, ese fichero está también abierto y es compartido por el hijo.

4. Haz otro programa que genere dos procesos hijos, uno de ellos debe alcanzar el estado zombie pronto y el padre, pasado un tiempo, debe ejecutar un *wait* y acaba. Para identificar al hijo que hace que el padre reanude la ejecución, recoge el parámetro del *exit* del proceso hijo y muéstralo en el terminal (tienes que utilizar la macro WEXITSTATUS). El segundo hijo debe permanecer activo durante un tiempo más largo, quedando huérfano. Comprueba con el comando *ps* lo que pasa con los tres procesos a lo largo del tiempo.

5. Al final del código de uno de los procesos hijo anteriores, cambia su imagen con una llamada a alguna de las funciones del grupo (*execl, execv, execle, execve, execlp, execvp*) para que ejecute otro código seguida de un *printf* que muestre un texto en la consola. Comprueba si se ejecuta dicho *printf*.

6. Comprueba con *ps* lo que sucede con un proceso zombie cuando su padre acaba la ejecución sin ejecutar el *wait*.

7. Visualiza el vídeo sobre la medición de tiempos de ejecución.

8. Haz un programa para calcular la **media** en **doble precisión** de las tangentes de las raíces cuadradas de los primeros 70.000.000 números naturales (considerándo que dichos números son ángulos en radianes) de forma que el cálculo se haga con dos procesos hijo trabajando **simultaneamente**. El primero debe hacer el cálculo sobre los números pares y el segundo sobre los impares. Cada uno de ellos debe mostrar su resultado en la consola y posteriormente copiarlo en un fichero auxiliar compartido. Debe haber una forma de identificar en el propio fichero quien escribe cada resultado. **Paralelamente**, un tercer hijo debe realizar el cálculo completo y almacenar el resultado en el fichero auxiliar. **Cuando acaben los dos primeros hijos**, un cuarto hijo debe calcular el resultado en base a ambas contribuciones, leyéndolas del fichero auxiliar, y añadir el resultado en ese mismo fichero. Todos los procesos hijo deben además escribir los tiempos de ejecución de sus cálculos en el mismo fichero compartido. Finalmente, **cuando todos acaben**, el padre debe calcular y mostrar la diferencia entre los resultados calculados de las dos formas (es decir, los obtenidos por los hijos tercero y cuarto) y mostrar los tiempos de ejecución de cada uno de los cuatro hijos. (**ENTREGABLE**).